Maschinelles Lernen (ML), Wintersemester 2026/27
Arbeitsgruppe Diskrete Systeme

Übung 3: Lineare Regression

Die lineare Regression ist das denkbar einfachste Regressionsmodell, und gerade deshalb ein idealer Einstieg, um zwei grundlegend verschiedene Wege kennenzulernen, wie ein ML-Modell „lernen“ kann: einmal analytisch über eine geschlossene Formel, einmal iterativ über den *Gradientenabstieg*. Während die geschlossene Lösung nur für wenige, einfache Modelle existiert, ist der Gradientenabstieg universell einsetzbar und bildet insbesondere die Grundlage für das Training neuronaler Netze in den späteren Übungen (7 ff.). Was Sie hier über Verlustfunktion, Gradient und Lernrate lernen, übertragen wir dort direkt auf sehr viel größere Modelle.

3.1 Das Modell und die Verlustfunktion

Wir betrachten den einfachsten Fall: ein einzelnes Merkmal x und eine Zielgröße y. Das lineare Regressionsmodell sagt

$$\hat{y} = w \cdot x + b \tag{1}$$

vorher, mit Steigung w und Achsenabschnitt (*bias*) b als zu lernenden Parametern. Gegeben sind n Trainingsbeispiele (x_i, y_i). Gesucht sind die Parameter w, b, die den mittleren quadratischen Fehler (*mean squared error*, MSE) zwischen Vorhersage $\hat{y}_i$ und tatsächlichem Wert y_i minimieren:

$$L(w,b) = \frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2 = \frac{1}{n}\sum_{i=1}^{n}(w \cdot x_i + b - y_i)^2 . \tag{2}$$

$L(w,b)$ heißt *Verlustfunktion* (*loss function*): Für jede Wahl von w und b liefert sie einen Skalar, der angibt, wie gut das Modell mit diesen Parametern die Trainingsdaten erklärt. Das Training eines Modells besteht also darin, (w,b) so zu wählen, dass $L(w,b)$ minimal wird. Für die lineare Regression mit MSE-Verlust gibt es dafür zwei grundverschiedene Wege.

3.2 Weg 1: Die geschlossene Lösung (Normalengleichung)

Da $L(w,b)$ in Gleichung (2) eine quadratische, konvexe Funktion der Parameter ist, besitzt sie genau ein globales Minimum, das sich direkt berechnen lässt: Man leitet L nach w und b ab, setzt beide Ableitungen gleich Null und löst das resultierende lineare Gleichungssystem. Praktischer ist die Matrixschreibweise mit dem sogenannten *Bias-Trick*: Man erweitert jeden Eingabevektor um eine konstante 1 (z. B. `X_b = np.c_[np.ones(n), X]`), sodass der Bias als zusätzlicher, mitgelernter Eintrag des Parametervektors $\theta = (b, w)^T$ erscheint. Dabei ist $X_b \in \mathbb{R}^{n\times 2}$ die um die Einsen-Spalte erweiterte Datenmatrix (eine Zeile pro Trainingsbeispiel), X_b^T ihre Transponierte und $\mathbf{y} \in \mathbb{R}^n$ der Vektor aller Zielwerte. Die Lösung, die L minimiert, ist dann durch die *Normalengleichung*

$$\theta = \left(X_b^T X_b\right)^{-1} X_b^T \mathbf{y} \tag{3}$$

gegeben. Dieser Weg liefert die exakte Lösung in einem einzigen Rechenschritt, ohne Iteration und ohne Lernrate. Der Nachteil: Die Matrixinversion in Gleichung (3) kostet für p Parameter $O(p^3)$ Rechenoperationen und wird bei vielen Parametern schnell unpraktikabel. Außerdem funktioniert dieser Weg nur, weil das Modell linear in den Parametern und der Verlust quadratisch ist – bei den meisten anderen Modellen, die wir im Laufe des Semesters kennenlernen, gilt mindestens eine der beiden Eigenschaften nicht mehr, sodass gar keine geschlossene Lösung dieser Art existiert.

1. Implementieren Sie Gleichung (3) selbst mit NumPy (Bias-Trick plus Matrixinversion, alternativ `np.linalg.lstsq` zum Vergleich).

3.3 Weg 2: Gradientenabstieg

Der *Gradientenabstieg* (*gradient descent*) löst dasselbe Minimierungsproblem iterativ und ganz ohne Matrixinversion, und funktioniert damit, im Gegensatz zur Normalengleichung, prinzipiell für beliebige (auch nicht-konvexe) Verlustfunktionen und beliebig viele Parameter. Die Idee: Man startet mit zufälligen Parametern und bewegt sich in jedem Schritt ein kleines Stück in die Richtung des *negativen* Gradienten der Verlustfunktion, also in die Richtung des steilsten Abstiegs. Der *Gradient* ist dabei einfach der Vektor aus den partiellen Ableitungen der Verlustfunktion nach allen Parametern, hier also $(\partial L/\partial w,\ \partial L/\partial b)$. Eine partielle Ableitung nach w beschreibt, wie stark sich L ändert, wenn man w ein kleines Stück verändert und b dabei konstant hält.

Herleitung der Ableitungen über die Kettenregel. In (2) steht w nicht direkt, sondern *innerhalb* der quadrierten Klammer. Die Verlustfunktion ist also eine verschachtelte Funktion. Für solche Fälle gibt es die *Kettenregel*: Ist $u(w)$ eine innere Funktion und wird sie quadriert, so gilt

$$\frac{\partial}{\partial w} u(w)^2 = \underbrace{2\,u(w)}_{\text{äußere Ableitung}} \cdot \underbrace{\frac{\partial u}{\partial w}}_{\text{innere Ableitung}},$$

kurz: „äußere Ableitung mal innere Ableitung“. Mit der inneren Funktion $u_i(w, b) = wx_i + b - y_i$ (dem Vorhersagefehler des i-ten Beispiels) und ihren Ableitungen $\partial u_i/\partial w = x_i$ sowie $\partial u_i/\partial b = 1$ folgt damit aus (2) unmittelbar

$$\frac{\partial L}{\partial w} = \frac{1}{n}\sum_{i=1}^{n} 2\,u_i \cdot \frac{\partial u_i}{\partial w} = \frac{2}{n}\sum_{i=1}^{n} (wx_i + b - y_i)\,x_i, \qquad \frac{\partial L}{\partial b} = \frac{2}{n}\sum_{i=1}^{n} (wx_i + b - y_i)\,. \tag{4}$$

Dieses Muster (äußere Ableitung des Losses mal innere Ableitung des Modells) ist genau dasselbe, das in Übung 8 beim Training neuronaler Netze zum Einsatz kommt; dort sind lediglich mehr Funktionen ineinander verschachtelt. Mit einer festen *Lernrate* η (*learning rate*) werden die Parameter in jeder Iteration (*Epoche*) aktualisiert:

$$w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w}, \qquad b \leftarrow b - \eta \cdot \frac{\partial L}{\partial b}. \tag{5}$$

Dies wird über eine feste Anzahl Epochen oder bis zur Konvergenz wiederholt. Die Lernrate η ist dabei entscheidend: Ist sie zu klein, konvergiert das Verfahren nur sehr langsam; ist sie zu groß, kann es über das Minimum „hinausschießen“ und divergieren oder oszillieren, statt sich dem Minimum anzunähern.

Öffnen und bearbeiten Sie das Notebook `ML_U03_Vorlage.ipynb`.

1. Implementieren Sie den Gradientenabstieg aus den Gleichungen (4) und (5) selbst: MSE-Verlust, Gradientenberechnung und eine Trainingsschleife über Epochen, die w und b iterativ aktualisiert und den Verlust je Epoche protokolliert. Lernrate und Epochenzahl sollen dabei als Parameter frei wählbar sein.
2. Vergleichen Sie die mit Gradientenabstieg gelernten Parameter mit der geschlossenen Lösung aus Aufgabe 2 (Werte von w, b sowie die resultierende Regressionsgerade gemeinsam mit den Datenpunkten geplottet).
3. Visualisieren Sie die Verlustfläche $L(w, b)$ als Contourplot über einem Gitter von (w, b)-Werten und zeichnen Sie den Verlauf Ihrer Gradientenabstiegs-Trajektorie hinein.
4. Experimentieren Sie mit zwei bis drei unterschiedlichen Lernraten (z. B. sehr klein, „passend“, zu groß) und beschreiben Sie anhand des Verlustverlaufs über die Epochen, welchen Einfluss die Lernrate auf die Konvergenz hat.

Optional: Validierung mit `scikit-learn`

Wer die eigene Implementierung zusätzlich absichern möchte, kann sie im Notebook optional gegen die Klasse `LinearRegression` aus `sklearn.linear_model` prüfen, und zwar auf einem echten Datensatz: `load_diabetes` aus `sklearn.datasets` mit einem einzelnen Merkmal, z. B. dem BMI-Wert. Stimmen die von `scikit-learn` gelernten Parameter mit denen Ihrer eigenen Normalengleichung bzw. Ihres Gradientenabstiegs überein?